INDUS INSTITUTE OF TECHNOLOGY & ENGINEERING

Indus University

Object Oriented Programming with UML (CE0316)

PROJECT REPORT

ELECTRICITY BILLING SYSTEM

Project Code: CPP29

Student Name: Rudraksh Madhur Mehta

Enrollment Number: IU2541230461

Branch: CSE Section: E

Faculty: Ms. Anjali Chopra

GitHub Repository: https://github.com/isagiyorichi/cpp-project

1. Project Title

Electricity Billing System

2. Student Details

Item	Details
Student Name	Rudraksh Madhur Mehta
Enrollment Number	IU2541230461
Branch and Section	CSE, Section E
Faculty	Ms. Anjali Chopra
Project Code	CPP29
Project Name	Electricity Billing System
Subject	Object Oriented Programming with UML (CE0316)
Institute	Indus Institute of Technology & Engineering, Indus University
GitHub Repository	https://github.com/isagiyorichi/cpp-project

3. Problem Statement

Electricity companies must make a bill for every customer every month. When this is done on paper or with simple notes, mistakes happen easily. The units can be calculated wrongly, a bill can be made twice, a payment can be forgotten, and the old records can be lost.

This project is a C++ program that solves this problem. It keeps the records of customers, meters and meter readings. It works out the bill amount from the units used and the price plan of the customer (Home plan or Shop plan). It also keeps track of which bills are paid and which are not paid. All the records are saved in files, so they are not lost when the program is closed.

4. Objectives

- To make a menu driven C++ program that manages customers, meters, readings and bills.
- To calculate the bill amount correctly from the units used and the price plan.
- To use the main ideas of object oriented programming: classes and objects, encapsulation, constructors and destructors, inheritance and runtime polymorphism.
- To use dynamic memory (new and delete[]) in a meaningful way.
- To save the records in files and load them again, so the data stays after the program is closed.

- To check the input and handle wrong values with exception handling.
- To keep the code in separate .h and .cpp files, and to explain the design with UML diagrams.

5. Features

The main features of the system are:

- Add, display, search, update and delete records.
- Make a monthly bill from a meter reading.
- Two price plans: Home plan and Shop plan.
- Pay a bill and change its status from Unpaid to Paid.
- Report with total units, total amount, and paid and unpaid bills.
- Saving and loading of data from text files.
- Checks on the input, with clear error messages.

Menu operations

No.	Menu option	What it does
1	Add Record	Adds a customer, a meter or a reading. The program checks that an ID is not used twice.
2	Display Records	Shows all customers, all meters, all readings or all bills.
3	Search Record	Finds a customer by ID and shows the meters and bills of that customer.
4	Update Record	Changes the name, address, phone or plan of a customer.
5	Delete Record	Deletes a customer, a meter or a reading, with safety checks.
6	Main Transaction	Makes a bill from a meter reading, or marks a bill as paid.
7	Report	Shows the totals of units and amounts, and the paid and unpaid bills.
8	Save / Load	Saves the records to files, or loads them from files.
9	Exit	Saves all the records and closes the program.

6. Classes and Responsibilities

The program has the following classes. Each class has its own .h file (the class description) and .cpp file (the code of the functions).

Class	Data members	Responsibility and important functions
Customer	customerId, name, address, phone, planType (1 = Home, 2 = Shop)	Keeps the details of one customer. The set functions check the values. Important functions: get and set functions, display().
Meter	meterId, customerId, location	Keeps the details of one meter and the customer who owns it. Important functions: get and set functions, display().
Reading	meterId, month, year, previousReading, currentReading	Keeps the meter numbers of one month. getUnitsUsed() gives the current reading minus the previous reading.
Bill	billId, customerId, meterId, month, year, units, amount, isPaid	Keeps one monthly bill. calculateAmount(Tariff*) asks the price plan for the amount. markAsPaid() changes the status to Paid.
Tariff	planName, fixedFee	Parent class of the price plans. It has the virtual function calculateAmount(units).
HomeTariff	Uses planName and fixedFee of the parent	Child of Tariff. Fixed fee Rs 50. The first 100 units cost Rs 4 each and the next units cost Rs 6 each.
ShopTariff	Uses planName and fixedFee of the parent	Child of Tariff. Fixed fee Rs 150. Every unit costs Rs 8.
FileManager	No data members	Saves the records to text files and loads them back. Important functions: saveCustomers(), saveMeters(), saveReadings(), saveBills(), the matching load functions, countLines(), splitLine().
BillingSystem	Dynamic arrays customers, meters, readings, bills, with their counts and capacities, and a FileManager object	The main system class. It does all the menu operations: addRecord(), displayRecords(), searchRecord(), updateRecord(), deleteRecord(), mainTransaction(), showReport(), saveData(), loadData().

The file main.cpp is short. It only makes the BillingSystem object, shows the menu and calls the functions of BillingSystem. All the work is done inside the classes.

7. OOP Concepts Used

7.1 Classes and Objects

A class is a plan and an object is a thing made from that plan. In this project, Customer, Meter, Reading, Bill and BillingSystem are classes. An object is made like this:

```
Customer c(1, "Rahul", "oak street", "1234567890", 1);
```

7.2 Encapsulation

The data members are private, so other code cannot change them directly. They can only be changed through get and set functions. The set functions check the value before saving it.

```
private:
   string phone;
public:
   void setPhone(string p);   // checks that the phone has 10 digits
```

7.3 Constructors and Destructor

Every entity class has a default constructor (no values), a parameterized constructor (with values) and a destructor. The BillingSystem constructor makes the arrays and its destructor frees them.

```
Customer();
Customer(int id, string n, string a, string p, int type);
~Customer();
```

7.4 Inheritance

Tariff is the parent class. HomeTariff and ShopTariff are child classes of Tariff. They get planName and fixedFee from the parent.

```
class HomeTariff : public Tariff { ... };
class ShopTariff : public Tariff { ... };
```

7.5 Runtime Polymorphism

The function calculateAmount() is virtual in Tariff and each child class writes its own version. In generateBill(), a Tariff pointer is made to point to a HomeTariff or a ShopTariff object, depending on the plan of the customer. When calculateAmount() is called, the program chooses the correct version while it is running.

```
Tariff* plan;
if (customers[c].getPlanType() == 1) plan = new HomeTariff();
else plan = new ShopTariff();
b.calculateAmount(plan);   // Home or Shop version runs
delete plan;
```

7.6 Dynamic Memory

The records are kept in arrays that are made with new. At the start, each array can hold 10 records. When an array is full, a function such as growCustomers() makes a new array that is

two times bigger, copies the old records, deletes the old array with delete[], and starts using the new array. The destructor of BillingSystem frees all the arrays. The purpose is that the program can keep any number of records, and it does not waste memory.

```
customers = new Customer[customerCapacity];
Customer* bigger = new Customer[newCapacity];
delete[] customers;
```

7.7 Validation and Exception Handling

The set functions and constructors check the values. If a value is wrong, they throw an error message. The functions of BillingSystem catch the error and show the message, so the program does not stop.

```
if (p.length() != 10) { throw "Phone must have exactly 10 digits"; }
try { ... } catch (const char* message) { cout << "Error: " << message << endl; }
```

7.8 Modular Files

Each class is written in its own pair of files, for example Customer.h and Customer.cpp. This keeps the code neat and easy to find.

8. File Handling

8.1 Data that is stored

The program stores four kinds of records in four text files in the Data folder.

File	Format of one line	Example
customers.txt	id|name|address|phone|plan	1|rudi|kelp street|1234565432|1
meters.txt	meterId|customerId|location	1|1|oak street
readings.txt	meterId|month|year|previous|current	1|5|2026|250|350
bills.txt	billId|customerId|meterId|month|year|units|amount|paid (1 or 0)	1|1|1|5|2026|100|450.00|1

Every record is one line, and the parts of the record are separated by the | sign.

8.2 Saving and loading

- **Loading:** When the program starts, loadData() counts the lines of each file with countLines(), makes an array that is big enough, and reads the records with the load functions of FileManager. A line that is wrong is skipped and a message is shown. If a file does not exist, the program starts with no records.
- **Saving:** When the user chooses Save in the menu, or Exit, saveData() writes all the records to the files with the save functions of FileManager.

8.3 Persistence testing

1. Run the program and add a customer, a meter and a reading. Make a bill.
2. Choose Exit (option 9). The program saves the data and closes.
3. Run the program again. The message Data loaded shows the number of records that were read.
4. Choose Display Records. The customer, meter, reading and bill are still there. This shows that the data is saved correctly.

9. UML Diagrams

9.1 Use Case Diagram

The Use Case Diagram shows the two actors, the Billing Clerk and the Customer, and what they can do with the system. The use case Make Bill includes the use case Calculate Amount.

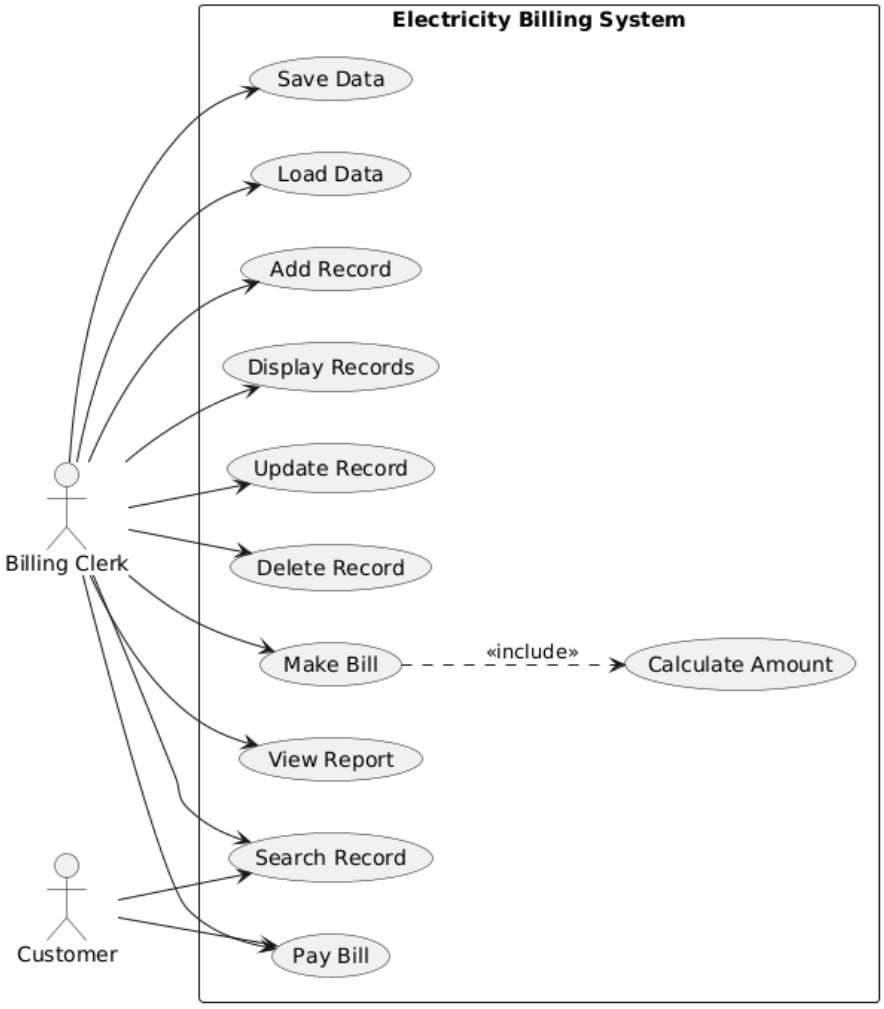


Figure 1: Use Case Diagram

9.2 Class Diagram

The Class Diagram shows the classes with their data members, member functions and visibility (- private, # protected, + public). Tariff is the parent of HomeTariff and ShopTariff. BillingSystem keeps the Customer, Meter, Reading and Bill records and uses FileManager to save and load them.

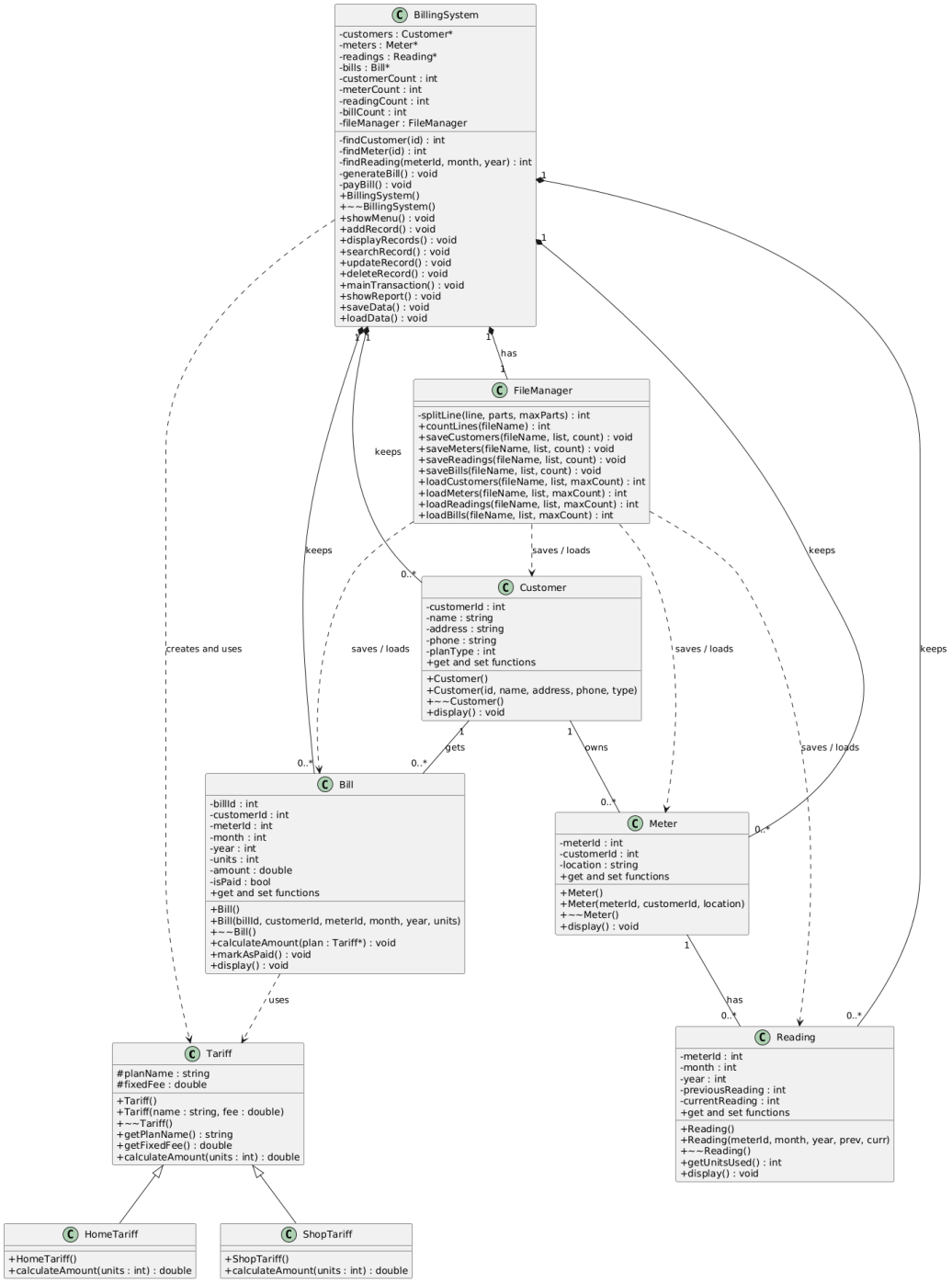


Figure 2: Class Diagram

9.3 Sequence Diagram 1: Make a Bill

This diagram shows the main transaction. The clerk chooses Main Transaction, the system finds the meter, the reading and the customer, makes the Bill, creates the correct Tariff plan, calculates the amount and shows the bill.

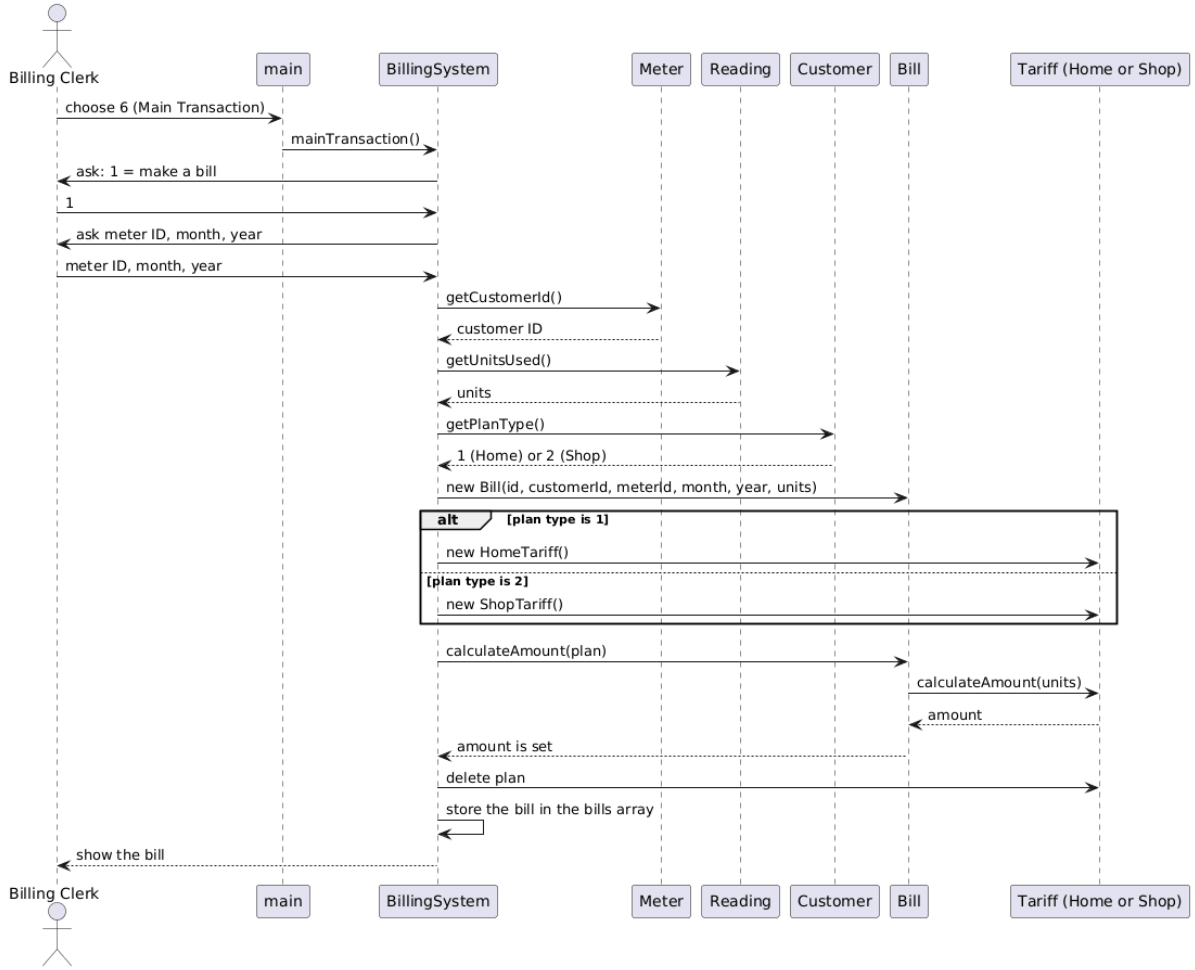


Figure 3: Sequence Diagram 1 (Make a Bill)

9.4 Sequence Diagram 2: Save Data

This diagram shows how the data is saved when the clerk chooses Exit. BillingSystem calls FileManager, and FileManager reads the values from the records and writes one line for each record to the data files.

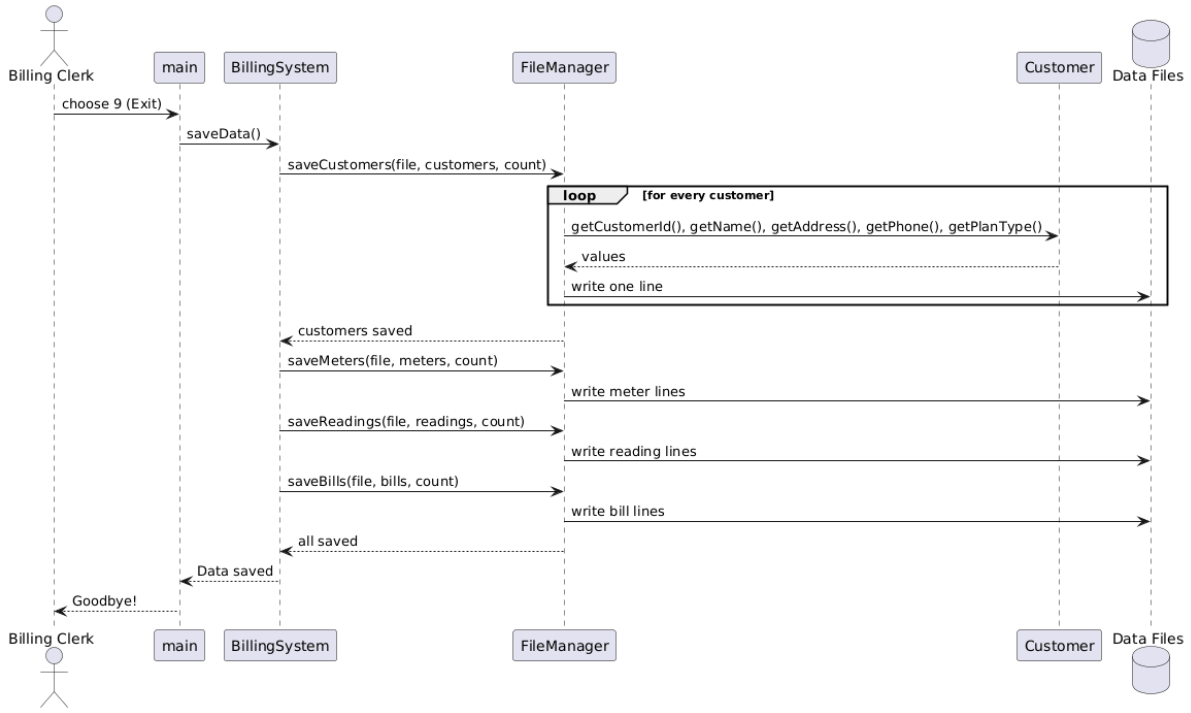


Figure 4: Sequence Diagram 2 (Save Data)

10. Program Workflow

The program works in the following way from start to exit:

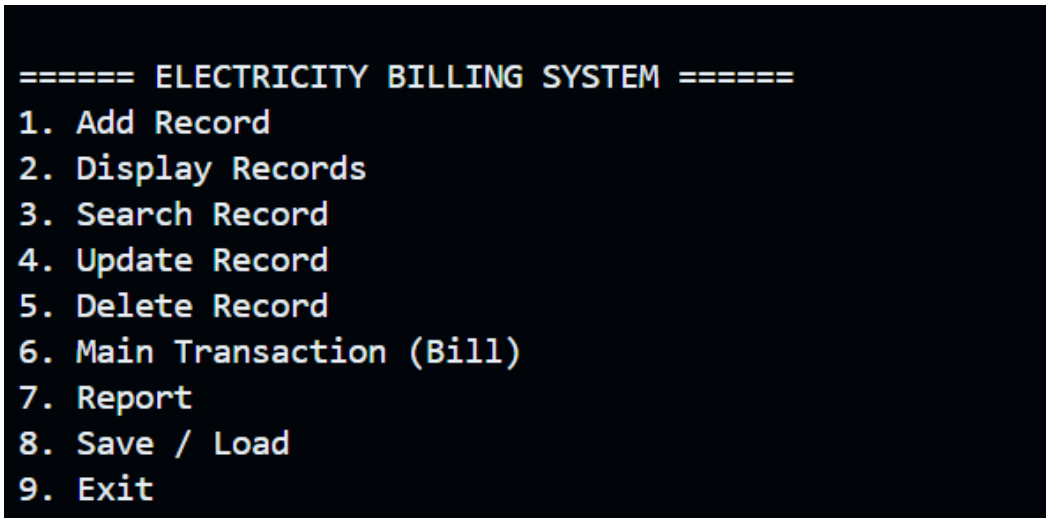
1. The program starts and main() makes the BillingSystem object. The constructor makes the arrays with new.
2. main() calls loadData(). The old records are read from the Data folder.
3. The main menu is shown and the user types a choice from 1 to 9.
4. main() calls the matching function of BillingSystem (add, display, search, update, delete, transaction, report, save or load).
5. BillingSystem does the work with the entity classes. For a bill, it uses the Reading, the Meter, the Customer and the Tariff plan, and then makes a Bill.
6. The menu is shown again. Steps 3 to 5 repeat until the user chooses Exit.
7. On Exit, saveData() saves all the records through FileManager.
8. The program ends. The destructor of BillingSystem frees the memory with delete[].

Flow in short

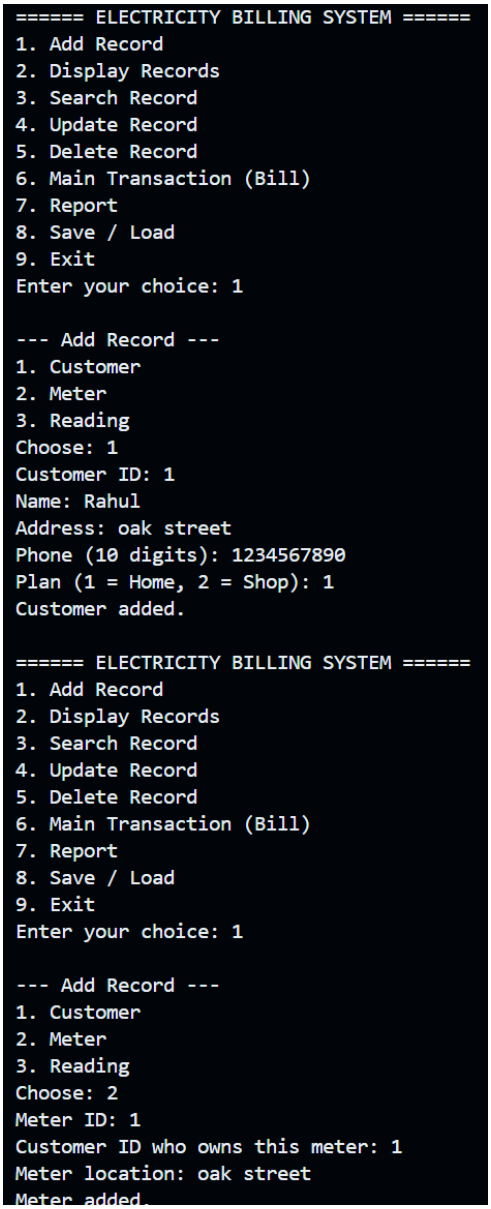
USER → MAIN MENU → BILLINGSYSTEM → CUSTOMER, METER, READING, BILL, TARIFF → MAKE BILL → FILEMANAGER → DATA FILES

11. Screenshots

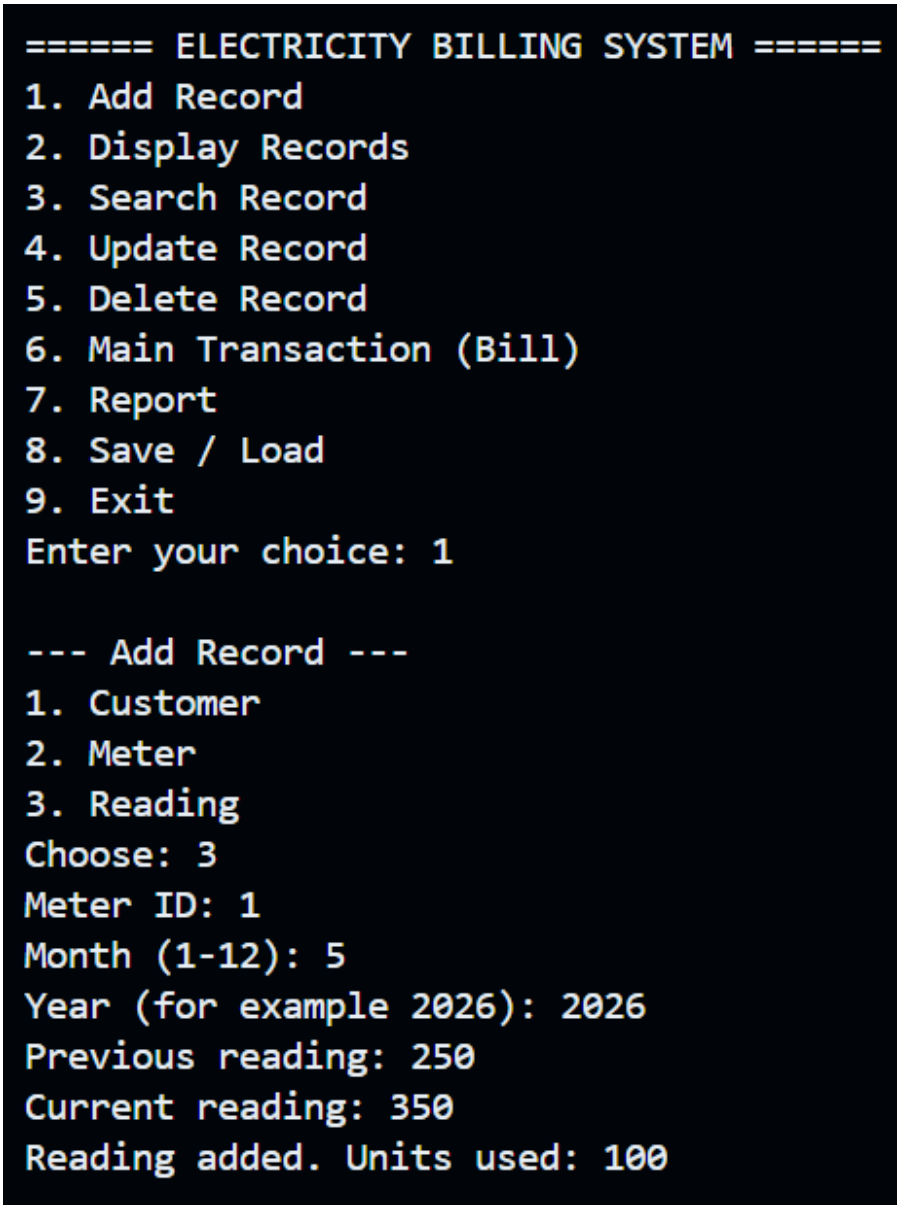
The following screenshots show the important operations and a successful run of the program.



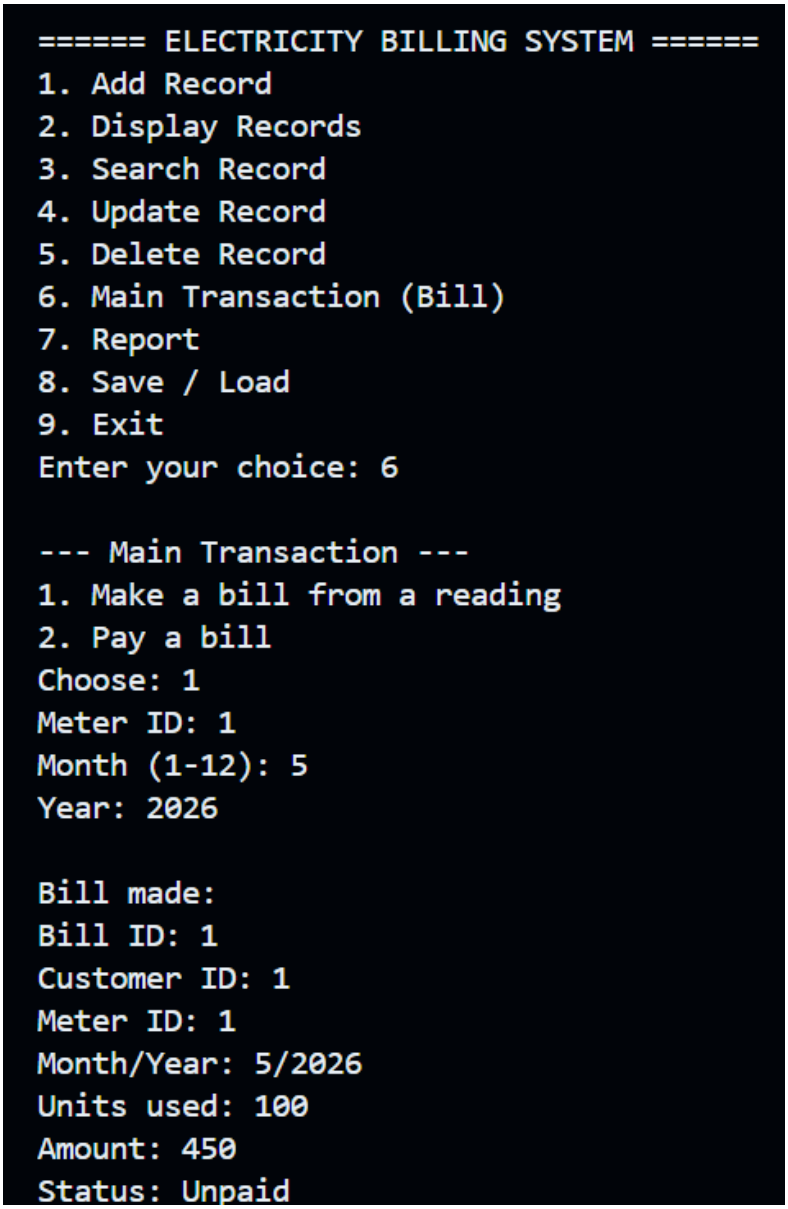
Screenshot 1: Main menu



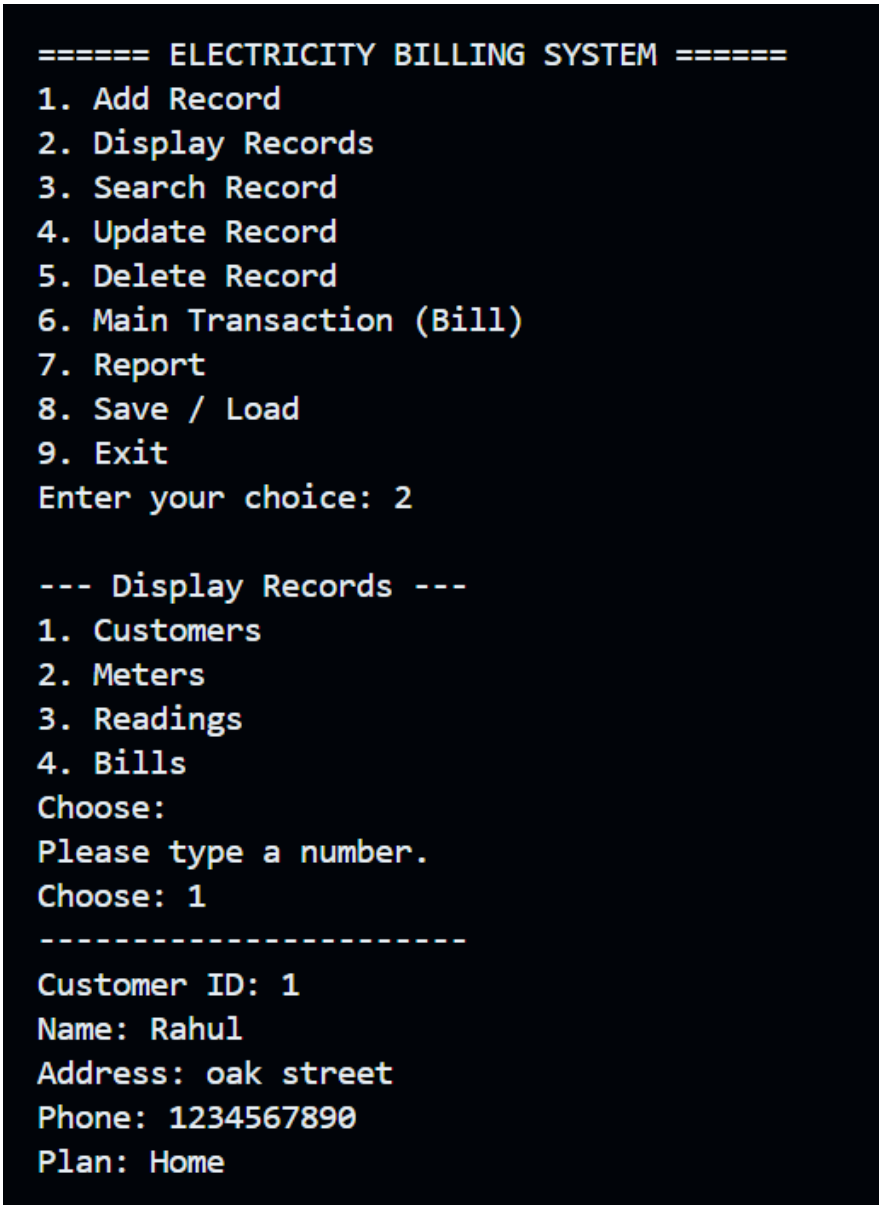
Screenshot 2: Adding a customer and a meter



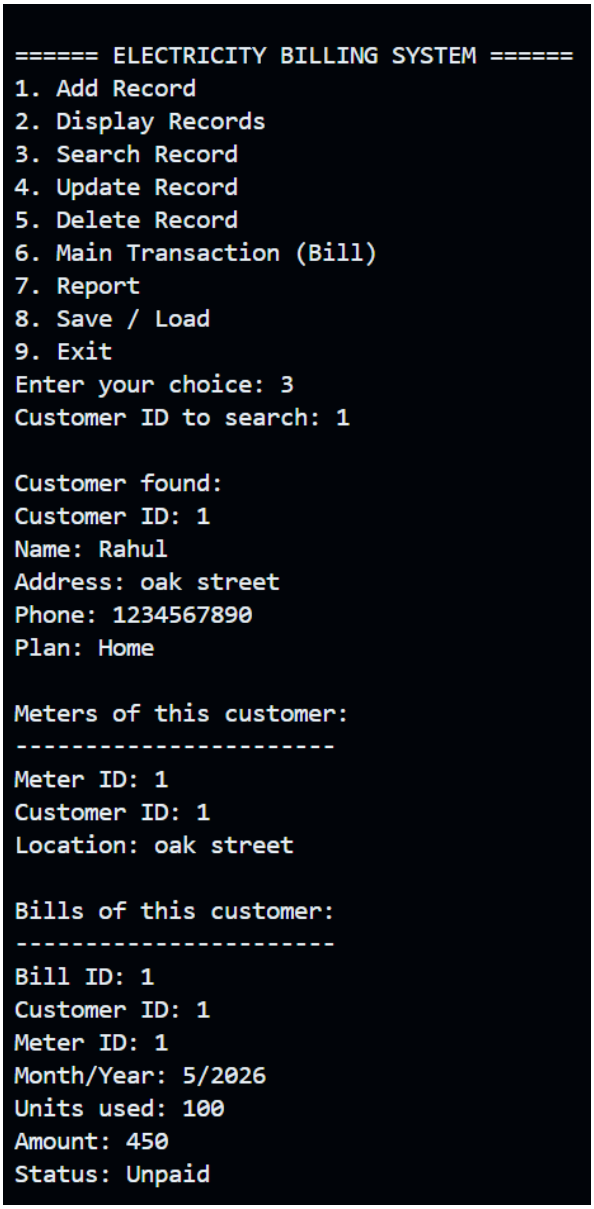
Screenshot 3: Adding a meter reading



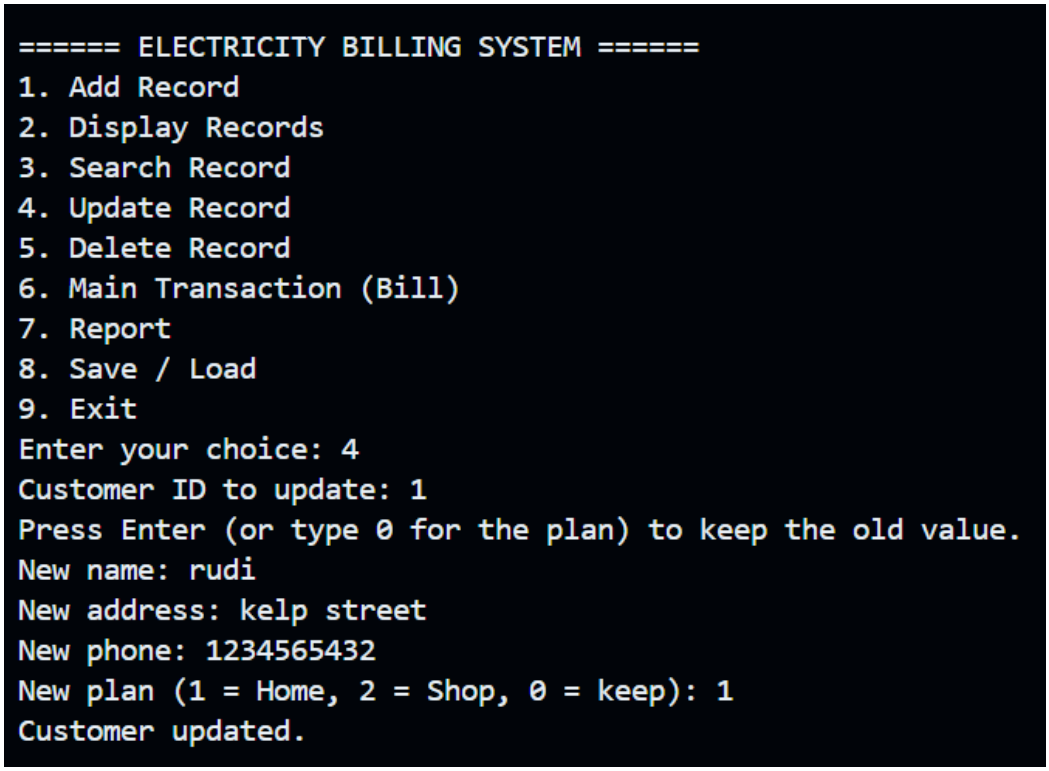
Screenshot 4: Making a bill (Main Transaction)



Screenshot 5: Display Records (customers) and the error message when no number is typed



Screenshot 6: Searching a customer (shows the customer, meter and bill)

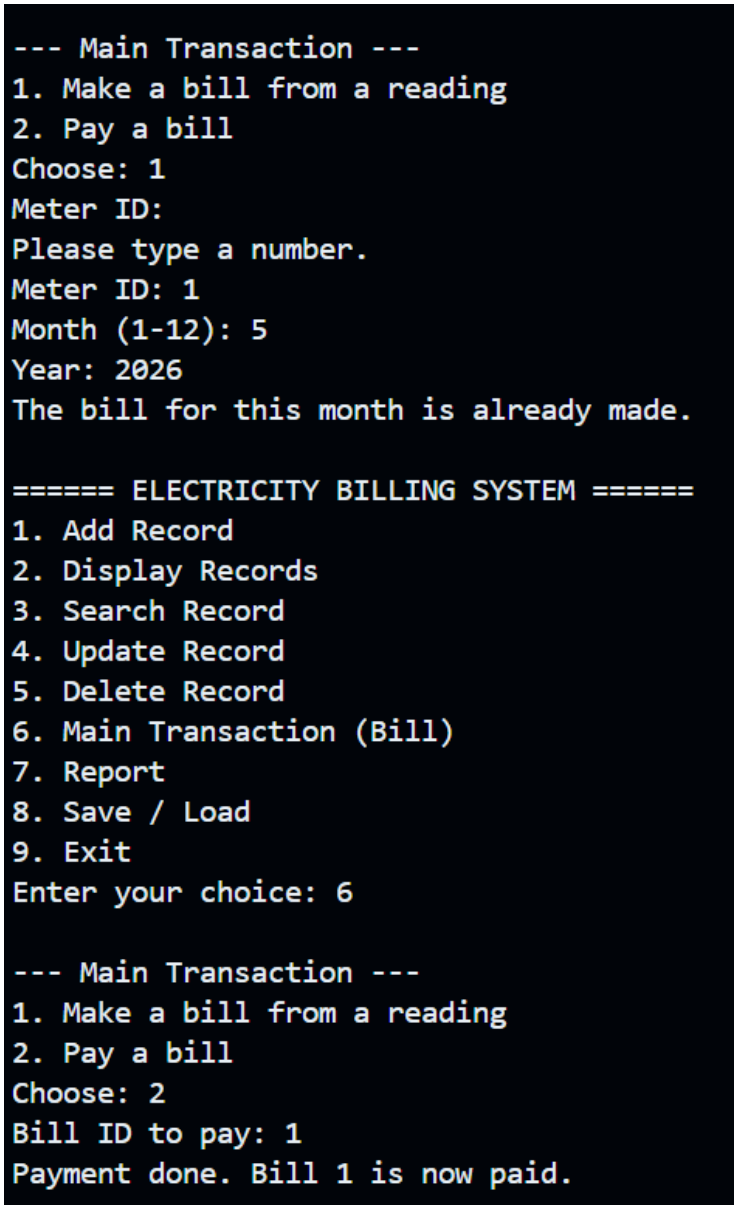


Screenshot 7: Updating a customer

```
====== ELECTRICITY BILLING SYSTEM ======
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction (Bill)
7. Report
8. Save / Load
9. Exit
Enter your choice: 7

========== REPORT ==========
Total customers: 1
Total meters: 1
Total readings: 1
Total bills: 1
Total units used: 100
Total amount billed: 450
Paid bills: 0 (amount 0)
Unpaid bills: 1 (amount 450)
```

Screenshot 8: Report with the bill unpaid



Screenshot 9: Error message for a bill that is already made, and paying a bill

```
========== REPORT ==========
Total customers: 1
Total meters: 1
Total readings: 1
Total bills: 1
Total units used: 100
Total amount billed: 450
Paid bills: 1 (amount 450)
Unpaid bills: 0 (amount 0)

====== ELECTRICITY BILLING SYSTEM ======
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction (Bill)
7. Report
8. Save / Load
9. Exit
Enter your choice: 8

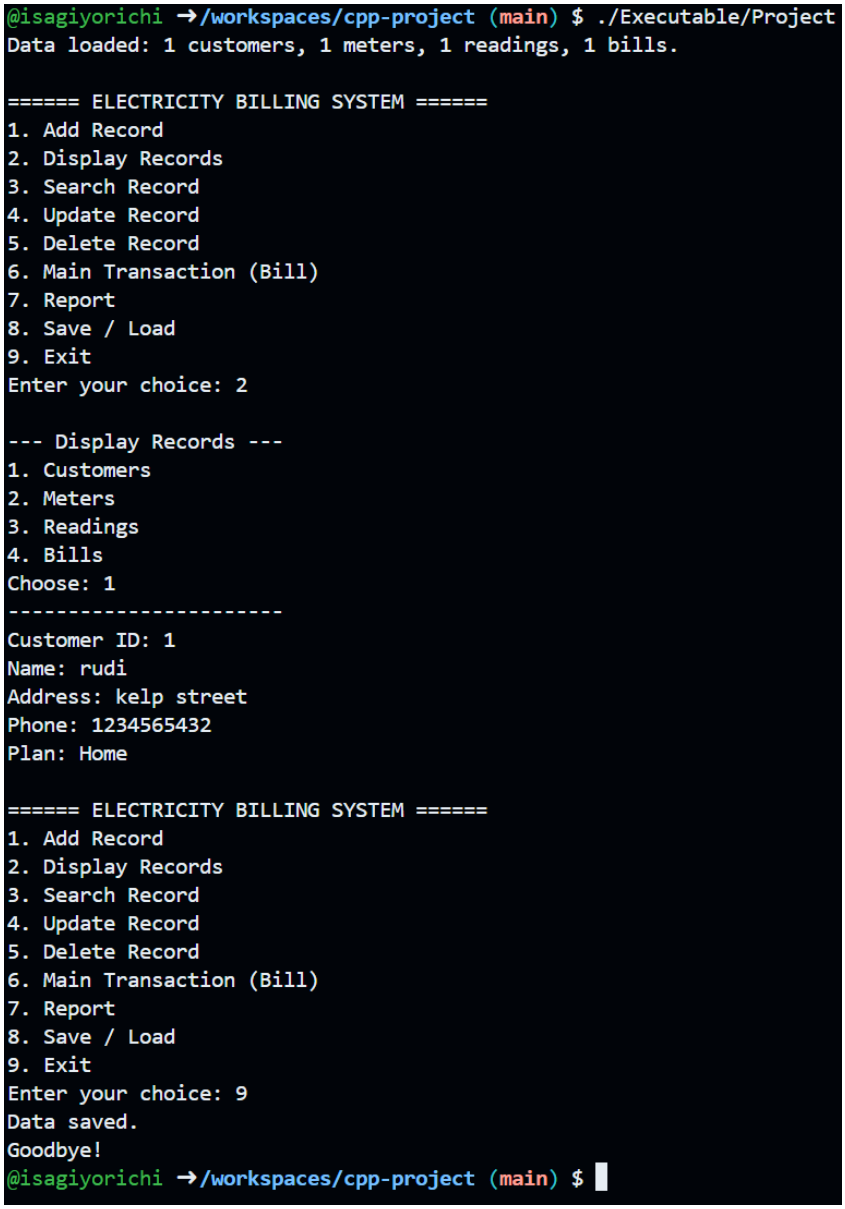
--- Save / Load ---
1. Save data to files
2. Load data from files (this replaces what is on screen now)
Choose: 1
Data saved.

====== ELECTRICITY BILLING SYSTEM ======
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction (Bill)
7. Report
8. Save / Load
9. Exit
Enter your choice:
Please type a number.
Enter your choice: 9
Data saved.
Goodbye!
```

Screenshot 10: Report after payment, saving the data and exit

```
   cat Data/customers.txt
   cat Data/bills.txt
bills.txt  customers.txt  meters.txt  readings.txt
1|rudi|kelp street|1234565432|1
1|1|1|5|2026|100|450.00|1
@isagiyorichi →/workspaces/cpp-project (main) $
```

Screenshot 11: Data files saved in the Data folder



Screenshot 12: Persistence test: the program is run again and the saved data is loaded

12. Testing

The following test cases were used to check the program. The Actual Output column shows the result that was seen when the program was run. The screenshot number of each result is written in brackets.

No.	Test case and input	Expected output	Actual output
1	Add a valid customer ID 1, Rahul, oak street, 1234567890, plan 1	Customer added.	Same as expected (Screenshot 2)
2	Add meter 1 for customer 1, location oak street	Meter added.	Same as expected (Screenshot 2)

No.	Test case and input	Expected output	Actual output
3	Add a reading for meter 1, month 5, year 2026, previous 250, current 350	Reading added. Units used: 100	Same as expected (Screenshot 3)
4	Make a bill for meter 1, month 5, year 2026 (Home plan)	Bill ID 1, Units used 100, Amount 450, Status Unpaid (100 x 4 + 50 fixed fee)	Same as expected (Screenshot 4)
5	Display records, choose Customers	The customer is shown with ID, name, address, phone and plan	Same as expected (Screenshot 5)
6	Press Enter without typing a number in a menu	Please type a number.	Same as expected (Screenshot 5)
7	Search customer ID 1	Shows the customer, the meter and the bill of the customer	Same as expected (Screenshot 6)
8	Update customer 1 with a new name, address and phone	Customer updated.	Same as expected (Screenshot 7)
9	Choose Report (option 7) with one unpaid bill	Total customers 1, meters 1, readings 1, bills 1, units 100, amount 450, Unpaid bills 1 (amount 450)	Same as expected (Screenshot 8)
10	Make the same bill again	The bill for this month is already made.	Same as expected (Screenshot 9)
11	Pay bill 1	Payment done. Bill 1 is now paid.	Same as expected (Screenshot 9)
12	Choose Report after the payment	Paid bills 1 (amount 450), Unpaid bills 0 (amount 0)	Same as expected (Screenshot 10)
13	Save the data (option 8, then 1) and Exit (option 9)	Data saved. then Goodbye!	Same as expected (Screenshot 10)
14	Look at the files in the Data folder	Files bills.txt, customers.txt, meters.txt and readings.txt, with one line for each record	Same as expected (Screenshot 11)
15	Run the program again and display the customers	Data loaded: 1 customers, 1 meters, 1 readings, 1 bills. The saved customer is shown again	Same as expected (Screenshot 12)

13. Challenges and Solutions

Challenge	Solution
The program could not save the files and showed the error that the file could not be opened.	The Data folder did not exist. Git does not keep empty folders, so the folder was missing. The folder was created with mkdir and a data file was saved inside it, so the folder is now kept in the repository.
The program must find the Data folder.	The files are opened with the path Data/..., so the program must be started from the main project folder. This is written in the README file.
The user can type wrong values, such as letters in place of numbers.	A function askNumber() keeps asking until the user types a proper number. The set functions check the values and throw an error message.
The number of records is not known in advance.	The arrays are made with new and they grow by two times when they are full. The old array is freed with delete[].
Wrong lines in a data file could stop the program when loading.	The load functions use try and catch. A wrong line is skipped and a message is shown.
The Home plan and Shop plan work out the amount in different ways.	Tariff has a virtual function calculateAmount(). Each child class writes its own version, and a Tariff pointer is used to call the correct one.
The diagram pictures were saved in the wrong folder (Source/Diagrams).	The pictures were moved to the Diagrams folder with the mv command, and the changes were pushed to GitHub.

14. Conclusion

This project made a working Electricity Billing System in C++. The program can add, display, search, update and delete records, make bills from meter readings, handle payments, show a report, and save and load the data from files.

While making the project, I learned how to use classes and objects, encapsulation, constructors and destructors, inheritance and runtime polymorphism in a real program. I also learned how to use dynamic memory with new and delete[], how to handle files, how to check input and handle errors, and how to explain a design with UML diagrams. I also learned to use Git and GitHub to keep the project safe.

In the future, the system can be made better by adding more price plans, a printed bill, a login for the clerk, and a way to search bills by month.

GitHub Repository

https://github.com/isagiyorichi/cpp-project